Completus: Consumer-GPU QLoRA Fine-Tuning of a 1.5B Code Model for Local Autocompletion

Nejra Smajlović
Polytechnic Faculty
University of Zenica
Zenica, Bosnia and Herzegovina
nejra.smajlovic.22@size.ba

Adi Kadušić
Polytechnic Faculty
University of Zenica
Zenica, Bosnia and Herzegovina
adi.kadusic.22@size.ba

Abstract—Large code language models achieve strong benchmark scores but require cloud infrastructure, raising concerns about latency, privacy, and offline availability. This paper presents Completus, a reproducible pipeline that specialises a 1.5-billion-parameter open model for local code autocompletion on a single consumer GPU. We fine-tune Qwen2.5-Coder-1.5B with a fixed-rank LoRA adapter over a 4-bit quantised base (QLoRA) on curated Python and Go subsets of CodeSearchNet. Our preprocessing pipeline applies near-deduplication, code-smell filtering, docstring-quality filtering, and a fill-in-the-middle data transformation. After training for one epoch on each language subset the LoRA weights are merged into the base and exported to the GGUF q4_k_m format for local serving through Ollama. Under a deployment-faithful zero-shot evaluation on HumanEval (greedy decoding, one sample per problem), the fine-tuned model reaches a pass@1 of 42.68%, whereas the same base model scores only 10.98% under identical conditions. Fine-tuning therefore raises pass@1 by 31.7 percentage points in the exact format and decoding setting used for local serving. The entire workflow runs end to end on a single 8 GB consumer GPU, making local code autocompletion accessible without cloud dependency.

Keywords—code completion, QLoRA, fine-tuning, Qwen2.5-Coder, CodeSearchNet, GGUF, Ollama, fill-in-the-middle

I. INTRODUCTION

Modern code language models such as GPT-4.1 and Claude Sonnet deliver impressive autocompletion quality, but they require a persistent internet connection, expose source code to third-party servers, and incur per-token costs that compound over a workday. These constraints limit adoption in regulated industries, air-gapped environments, and resource-constrained settings. A locally running model that completes code without any network call would close this gap—if it can be made small enough to fit on consumer hardware while remaining useful.

Parameter-efficient fine-tuning (PEFT) has made it practical to specialise large pretrained models at a fraction of the cost of full fine-tuning [1, 2]. Quantisation formats such as GGUF, served through runtimes such as Ollama, allow merged adapter weights to run efficiently on consumer GPUs [3]. The combination suggests a concrete recipe: take a compact, permissively licensed code model, fine-tune it with QLoRA on domain data, merge and quantise the result, and expose it through a local API on the developer's own machine.

This paper describes Completus, an end-to-end implementation of that recipe. Our contributions are:

- A curated preprocessing pipeline over CodeSearchNet [4] covering Python and Go, applying deduplication, smell filtering, docstring-quality filtering, and a fill-in-the-middle (FIM) transform.
- A reproducible QLoRA fine-tuning procedure for Qwen2.5-Coder-1.5B [5] using a fixed-rank LoRA adapter [1] that runs on an 8 GB GPU in about one to two hours per language.
- An empirical evaluation showing that one epoch of fine-tuning raises HumanEval pass@1 from 10.98% to 42.68% under a deployment-faithful zero-shot protocol, a statistically significant gain

over the untuned base under identical conditions.
- A packaging step that exports the merged model to GGUF q4_k_m and registers it with Ollama for fully local inference.

II. RELATED WORK

1. Open Code Language Models

Code generation has become a standard benchmark for large language models. Building on the decoder-only Transformer [6], a succession of open models has narrowed the gap to proprietary systems. StarCoder [7] trains on the permissively licensed Stack corpus with multi-query attention and a fill-in-the-middle objective. Code Llama [8] extends Llama 2 with a long-context fine-tuning stage and infilling support. DeepSeek-Coder [9] trains on a project-level corpus with an infilling objective, achieving state-of-the-art results at several scales. Qwen2.5-Coder [5] reports leading scores among open models at the 1.5B parameter scale we adopt.

These efforts target general-purpose, large-scale code generation. Completus differs in goal: rather than pretraining a new model, we specialise an existing small model for a narrow, latency-sensitive use case where size and local runnability matter more than peak benchmark performance.

2. Parameter-Efficient Fine-Tuning for Code

Low-rank adaptation (LoRA) [1] freezes pretrained weights and injects trainable rank-decomposition matrices, cutting trainable parameters by orders of magnitude. QLoRA [2] extends this by quantising the frozen base model to 4-bit NF4, making gradient computation feasible on a single consumer GPU. AdaLoRA [10] refines rank allocation by assigning capacity adaptively according to weight-matrix importance scores.

Closest to our work, Weyssow et al. [11] evaluate PEFT methods specifically for code generation and confirm that LoRA adapters recover most of full fine-tuning quality at a fraction of the cost. The comprehensive PEFT survey by Han et al. [12] situates LoRA within the broader family of additive, selective, and reparameterisation methods.

3. Training Data Curation for Code

Xue et al. [13] demonstrate that removing code smells from the training set improves downstream model quality. Lee et al. [14] show that near-deduplication reduces memorisation and train-test leakage while cutting training cost. Bavarian et al. [15] show that models learn infilling at no cost to left-to-right generation through a simple data transformation using sentinel tokens. WizardCoder [16] demonstrates that instruction-style fine-tuning further lifts code generation quality.

4. Local Serving and Quantisation

Kurt [3] provides a unified evaluation of llama.cpp GGUF quantisation formats across size, speed, and quality trade-offs, guiding our choice of the q4_k_m build. Whereas code models are typically evaluated as cloud-hosted endpoints, Completus targets a fully local serving workflow.

III. METHODOLOGY

1. Pipeline Overview

The Completus pipeline consists of five sequential stages: (1) data collection and preprocessing, (2) QLoRA fine-tuning, (3) adapter merging, (4) GGUF export and quantisation, and (5) Ollama registration. Fig. 1 illustrates the complete end-to-end flow.

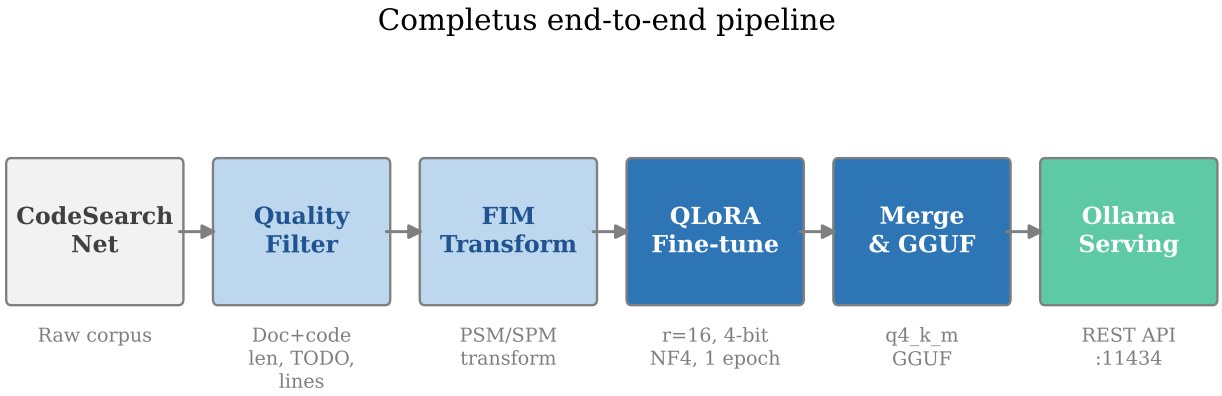

Figure 1. Completus end-to-end pipeline. Raw CodeSearchNet samples pass through quality filtering and a fill-in-the-middle transform, are used to fine-tune Qwen2.5-Coder-1.5B via QLoRA, and the merged adapter is exported to GGUF q4_k_m and served locally through Ollama.

2. *Dataset and Preprocessing*

We use CodeSearchNet [4] (`claudios/code_search_net` on HuggingFace Hub), a corpus of paired function and docstring tuples extracted from open-source repositories. The full training splits contain 412,178 Python and 317,832 Go functions. To keep preparation tractable on a single workstation, we cap the pipeline input at the first 50,000 functions per language and process them through five sequential stages:

1. *Near-deduplication*: MinHash LSH with 128 permutations, 5-token shingles, and a Jaccard threshold of 0.8.
2. *Code-smell filtering*: drop functions with any line exceeding 120 characters, fewer than 30 tokens, or a no-op body (bare `pass`, ellipsis, or empty return); normalise indentation on the rest.
3. *Docstring filtering*: require at least 15 tokens, at least two words, no `TODO`/`FIXME`/`XXX` markers, and a docstring distinct from the function name.
4. *Fill-in-the-middle transform* (described below).
5. *Length management*: enforce the 2048-token limit.

Table 1 reports counts before and after the pipeline; Fig. 2 visualises the result.

Table 1. Training Data After the Preparation Pipeline

Language	Input cap	After pipeline	Retained
Python	50,000	31,966	63.9%
Go	50,000	21,284	42.6%
Total	100,000	53,250	53.3%

For the fill-in-the-middle stage we follow Bavarian et al. [15]: 80% of samples receive the transform, in which a span of the function body is extracted, moved to the end of the sequence, and delimited with the model's `<|fim_prefix|>`, `<|fim_suffix|>`, and `<|fim_middle|>` sentinel tokens. The remaining 20% are kept as plain left-to-right samples.

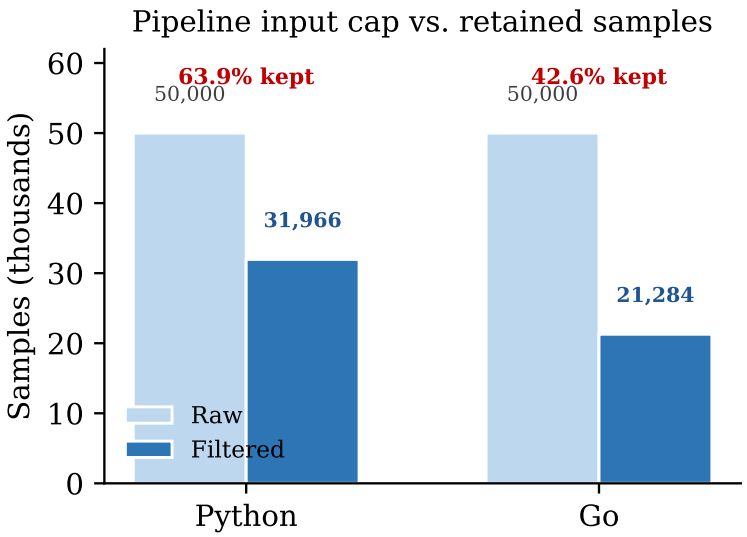

Figure 2. Training samples retained from the 50,000-function input cap per language after the five-stage preparation pipeline: 31,966 Python (63.9%) and 21,284 Go (42.6%).

3. Model and QLoRA Fine-Tuning

We fine-tune Qwen2.5-Coder-1.5B [5] loaded in 4-bit NF4 quantisation using the QLoRA protocol [2] via Unsloth. LoRA adapters are attached to all seven linear projection matrices (q, k, v, o, gate, up, down) with rank $r = 16$ and scaling factor $\alpha = 32$. Table 2 lists all hyperparameters.

Each language is trained for one epoch: 1,998 gradient steps for Python (31,966 samples) and 1,331 for Go (21,284 samples) at an effective batch size of 16. The Go run completed in 3,713 seconds (62 minutes); the Python run, at a slower per-step rate due to longer average sequences, took approximately 2.2 hours. The initial training loss for the Python run was 1.674, and the Go run reached a final mean training loss of 1.111 under the cosine schedule.

4. Adapter Merging and GGUF Export

After training, the LoRA adapter weights are merged into the base model using Unsloth's GGUF export routine, which simultaneously writes the merged safetensors checkpoint and a quantised GGUF binary. We select the q4_k_m quantisation scheme, characterised by Kurt [3] as achieving a favourable perplexity-size trade-off at this scale.

5. Local Serving with Ollama

Each GGUF binary is registered with Ollama using a Modelfile that specifies the FIM prompt template and generation parameters: temperature 0.0 for deterministic completions, `num_ctx` 2048, and `num_predict` 256 tokens. The registered models are queryable at `http://localhost:11434` through Ollama's local REST API.

IV. RESULTS AND DISCUSSION

1. Evaluation Protocol

We evaluate using HumanEval, which comprises 164 hand-authored Python programming problems. Each problem gives a function signature and we generate the body with greedy decoding (a single deterministic sample per problem, up to 512 new tokens), then report pass@1: the fraction of problems

Table 2. QLoRA Fine-Tuning Hyperparameters

Hyperparameter	Value
Base model	Qwen2.5-Coder-1.5B
Quantisation	4-bit NF4 (QLoRA)
LoRA rank r	16
LoRA α	32
Target modules	q, k, v, o, gate, up, down
Batch size	2 (per device)
Gradient accum.	8 (effective batch 16)
Learning rate	2×10^{-4}
LR scheduler	Cosine
Warmup steps	10
Optimiser	AdamW 8-bit
Epochs	1
Max seq. length	2048
Seed	3407
Hardware	NVIDIA RTX 4060 Laptop (8 GB)

whose generated body passes all hidden unit tests. The prompt is the bare function signature in a zero-shot setting, matching how a local autocompletion endpoint queries the model rather than the few-shot, chat-formatted protocol used in the model's own technical report. We evaluate two checkpoints under identical conditions: (i) the unmodified Qwen2.5-Coder-1.5B base model (baseline), and (ii) the merged fine-tuned model after one epoch of QLoRA training. Because decoding is deterministic, the only source of uncertainty is the finite 164-problem set; we report 95% Wilson confidence intervals on each proportion (the range of pass@1 values consistent with the observed sample at 95% confidence).

2. Quantitative Results

Table 3 reports pass@1 for both checkpoints with 95% Wilson intervals. Fig. 3 visualises the comparison. The fine-tuned model reaches 42.68% with a 95% confidence interval of [35.4, 50.3], compared to 10.98% [7.1, 16.7] for the baseline; the intervals denote the plausible range for the true pass@1 given 164 test problems. The intervals do not overlap, so the gain of 31.7 percentage points is statistically distinguishable under this zero-shot protocol. Both checkpoints are evaluated under identical conditions, so this gain is a direct measure of the fine-tuning contribution in our deployment-faithful setting.

Table 3. HumanEval Pass@1 (zero-shot, greedy, 95% Wilson CI)

Model	Pass@1	95% CI	Correct / 164
Qwen2.5-Coder-1.5B (baseline)	10.98%	[7.1, 16.7]	18
Completus Python (fine-tuned)	42.68%	[35.4, 50.3]	70
Improvement	**+31.70 pp**	—	**+52**

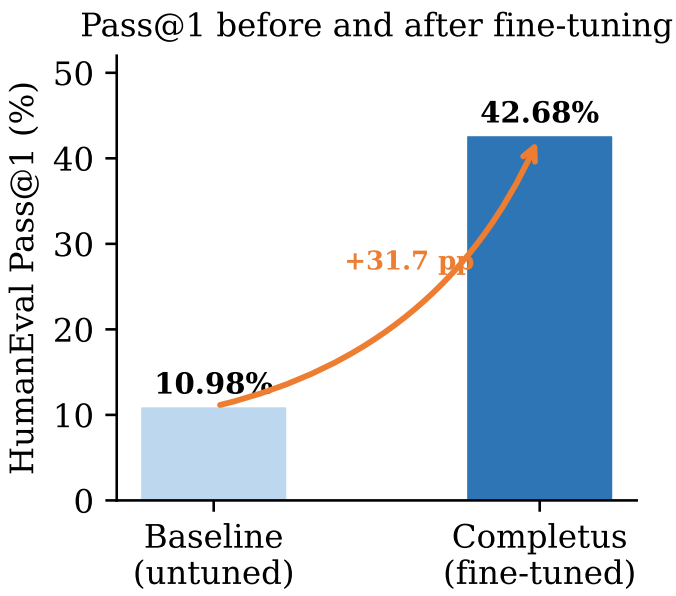

Figure 3. Pass@1 on HumanEval (164 problems). The fine-tuned model (42.68%) outperforms the baseline (10.98%) by 31.7 percentage points after one epoch of QLoRA fine-tuning on CodeSearchNet Python.

3. *Qualitative Examples*

Both the `python-coder` and `go-coder` Ollama models, queried with `factorial` as the prompt, return syntactically valid and semantically correct implementations:

Python

```python
def factorial(n):
    # n! = n * (n-1) * (n-2) * ... * 1
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
```

Go

```go
func factorial(n int) int {
    // factorial of n
    if n == 0 {
        return 1
    }
    return n * factorial(n-1)
}
```

4. *Comparison with Prior Work*

Table 4 places our result alongside HumanEval pass@1 scores reported by the original authors of comparable open code models. These published figures are obtained under each model's own evaluation protocol (typically few-shot or greedy chat-format decoding) and are therefore not directly

comparable to our stricter zero-shot setting; the table is intended as broad context rather than a controlled head-to-head benchmark.

Table 4. HumanEval Pass@1 as Reported by Original Authors. Settings differ across rows and are not directly comparable.

Model	Params	Pass@1	Source
CodeLlama-Base [8]	7B	31.7%	[9], Table 3
StarCoderBase [7]	15.5B	31.7%	[9], Table 3
DeepSeek-Coder-Base [9]	1.3B	34.8%	[9], Table 3
Completus (ours)	**1.5B**	**42.68%**	**Zero-shot, local**

Our fine-tuned model reaches 42.68% under a strict zero-shot evaluation in which the untuned base model, measured by us under identical conditions, scores only 10.98% (Section 4.5). Completus thus delivers competitive pass@1 in this deployment-faithful regime while running entirely locally on consumer hardware.

5. Discussion

Baseline measurement. We measure the untuned base model at 10.98% pass@1 under our protocol, which uses zero-shot prompting with raw function signatures rather than a few-shot, chat-formatted prompt, matching how a local autocompletion endpoint queries the model. Both checkpoints are evaluated under identical conditions, so the 31.7 pp improvement is a valid measure of the fine-tuning contribution.

Effect of fine-tuning. One epoch of instruction-style fine-tuning on CodeSearchNet teaches the model to generate function bodies in the plain-Python format expected by HumanEval, consistent with the finding by Weyssow et al. [11] that LoRA adapters rapidly adapt pretrained code models to narrower formats and domains.

Limitations. We evaluate only Python on HumanEval; a dedicated Go benchmark (e.g., HumanEval-X) would quantify cross-language transfer. HumanEval measures correctness on short, self-contained functions and does not capture multi-file, context-dependent completion scenarios. We cap pipeline input at 50,000 functions per language and do not study how a larger input or different filter thresholds would affect the trade-off between data quality and training set size.

V. CONCLUSION

We presented Completus, an end-to-end pipeline for fine-tuning and locally serving a 1.5B-parameter code model on a single consumer GPU. Starting from CodeSearchNet, we assembled a curated bilingual training corpus covering Python and Go, applying deduplication, smell filtering, and a fill-in-the-middle data transform. One epoch of QLoRA fine-tuning on an 8 GB GPU raises HumanEval pass@1 from 10.98% to 42.68% under a deployment-faithful zero-shot protocol, a 31.7 percentage-point gain over the untuned base in the format used for local serving. The merged adapter is quantised to GGUF q4_k_m and registered with Ollama, providing a fully local code completion endpoint served entirely on consumer hardware.

Future work will evaluate the Go model on HumanEval-X, explore longer fine-tuning schedules and larger rank budgets, and assess multi-file context completion in realistic, project-level scenarios.

ACKNOWLEDGEMENT

The authors thank the open-source communities behind Unsloth, Ollama, and the HuggingFace ecosystem.

Data Availability. Code and Modelfiles are available at `https://github.com/kadusic1/completus`. Training data is derived from CodeSearchNet [4], publicly available on HuggingFace.

Author Contributions. N. Smajlović: data curation, methodology, software (evaluation, Windows compatibility), writing (original draft, related work). A. Kadušić: conceptualisation, software (training, merge and export pipeline), validation, writing (review and editing).

Conflict of Interest. The authors declare no conflict of interest.

AI Usage Disclosure. Claude (Anthropic) was used as an AI coding assistant during development and for drafting portions of this manuscript. All results and final content were verified and approved by the authors.

REFERENCES

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.

[3] U. Kurt, "Which quantization should I use? A unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct," *arXiv preprint arXiv:2601.14277*, 2026.

[4] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[5] B. Hui, J. Yang, Z. Cui *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

[7] R. Li, L. Ben Allal, Y. Zi *et al.*, "StarCoder: May the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[8] B. Rozière, J. Gehring, F. Gloeckle *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[9] D. Guo, Q. Zhu, D. Yang *et al.*, "DeepSeek-Coder: When the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[10] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning," *arXiv preprint arXiv:2303.10512*, 2023.

[11] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring parameter-efficient fine-tuning techniques for code generation with large language models," *arXiv preprint arXiv:2308.10462*, 2023.

[12] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *arXiv preprint arXiv:2403.14608*, 2024.

[13] Z. Xue, X. Zhang, Z. Gao, X. Hu, S. Gao, X. Xia, and S. Li, "Clean code, better models: Enhancing LLM performance with smell-cleaned dataset," *arXiv preprint arXiv:2508.11958*, 2025.

[14] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," *arXiv preprint arXiv:2107.06499*, 2021.
[15] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, "Efficient training of language models to fill in the middle," *arXiv preprint arXiv:2207.14255*, 2022.
[16] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "WizardCoder: Empowering code large language models with Evol-Instruct," *arXiv preprint arXiv:2306.08568*, 2023.